Tipos de datos

Tipos numéricos

- **Entero** → int (%d)
- **Entero (de mayor precisión)** → long (%ld)
- **Real** → float (%f)
- **Real (de mayor precisión)** → double (%lf)

Tipo texto

- **Carácter** → char (%c)
  (en c no exist el tipo string. Una cadena de texto es un una lista de char acabada en '\0'. Se puede leer con %s).

Tipo lógico

- **Booleano** → bool (%d, se lee como 1 o 0)
  (requiere #include <stdbool.h>)

Tipo sin valor

- **Vacío** → void
  (se usa en funciones que no devuelven valor)

Tipo de dato estructurado

- **Tupla** → struct
  (Puede emplear cualquier tipo ya definido)

```
struct Palabra {
    char caracteres[50];
    int longitud;
} Palabra;
```

Fichero

- **Fichero** → FILE
  (se suele usar referenciado como un puntero)

Definición de variables

Para **definir una variable** usaremos la sentencia: “[tipo] [nombreVariable];”

```
int contador;
```

También podemos emplear “[tipo] [nombreVariable] = [valor];” si queremos **inicializar** dicha variable mientras la definimos.

```
char caracter = 'a';
```

En el caso de que queramos hacer una **tabla** (para por ejemplo hacer un string), tendríamos que usar “[tipo] [nombreVariable][tamaño];”. El tamaño debe ir entre corchetes.

```
char cadena[10];
```

Para definir una variable que sea un **puntero**, deberemos usar “[tipo] *[nombreVariable];”

```
int *p;
```

Punteros, aspectos clave

Para definir una **variable de tipo puntero** usamos “*”

```
char *caracter;
```

Para **obtener la dirección de memoria** de una variable usamos “&”. Especialmente útil a la hora de llamar a acciones con parámetros de e/s o salida.

```
int numero;
scanf("%d", &numero);
```

Para **acceder al valor en memoria** a partir de una dirección, usamos “*”. Útil para llamar a acciones con parámetros de entrada.

```
int numero = 10;
int *puntero = &numero;
printf("Valor usando puntero: %d\n", *puntero);
```

Funciones

Para **definir una función**, debemos usar:

```
[tipo_retorno] [nombre_funcion]([parámetros_entre_comas_tipados]) {
  // código
```

```
    return [valor]; // si no es void
}
```

```
void saludar() {
    printf("Hola\n");
}

int sumar(int a, int b) {
    return a + b;
}
```

Para **llamar a una función**, usaremos su nombre y después pondremos sus parámetros. OJO, que deben de ser del mismo tipo!!!! (atentos a los punteros):

```
[nombre_funcion]([parámetros_entre_comas]);
```

```
saludar();

int resultado = sumar(3, 4);
```

Nuestro **punto de entrada** de nuestro código siempre será una función "main" con parámetros vacíos (que suele devolver int):

```
[tipo] main() {
    // codigo
    return [valor]; // si no es void
}
```

```
int main() {
    printf("Hola mundo\n");
    return 0;
}
```

Funciones especiales

Funciones para **interactuar con la consola** (requieren #include <stdio.h>):

- **Escribir** en consola: printf([textoConFormato], [listaVariablesFormato]);
    - El texto con formato definirá cómo se va a imprimir por pantalla la información. PE: "%d %d" es "numeroEntero numeroEntero"
    - Hay que emplear los formatos de cada tipo de variable.
    - Se deben especificar tantas variables como sea necesario para cumplimentar el textoConFormato.

```
int edad = 20;
int *p = &edad;
printf("%d", *p);
printf("%d", edad);
```

- **Leer** de consola: scanf([textoConFormato], [listaDireccionesVariablesFormato]);
  - El texto con formato definirá cómo se va a leer de pantalla la información. PE: “%d %d” es “numeroEntero numeroEntero”
  - Hay que emplear los formatos de cada tipo de variable.
  - Se deben especificar tantas variables como sea necesario para cumplimentar el textoConFormato. Además estas variables deberán ser direcciones de memoria.

```
float precio;
scanf("%f", &precio);
```

- **Caracteres especiales**:
  - ‘\n’ salto de línea
  - ‘\t’ tabulación
  - ‘\\’ contra barra
  - ‘\’’ comilla simple
  - ‘\”’ comilla doble

Funciones para operar con **ficheros**:

- **Apertura**: fopen([rutaArchivo], [modo]);
  - Acepta tanto rutas relativas como absolutas
  - Modos:
    - Lectura “r”
    - Escritura “w”
    - Añadir “a”

```
FILE *f;
f = fopen("archivo.txt", "r");
```

- **Lectura:**
  - fscanf([variableFichero], [textoConFormato], [dirVariable]);
    - Funciona como scanf
    - Requiere de tener el fichero abierto
    - La variable de salida debe ser una dirección de memoria

```
int x;
fscanf(f, "%d", &x);
```

- fgets([variableString], [longitudMaxima], [variableFichero]);
    - Carga en la variable string (char[]) todo el texto que haya en el fichero hasta llegar a la longitud, o a salto de línea.
    - El fichero debe estar abierto

```
char linea[100];
fgets(linea, 100, f);
```

- **Escritura:** fprintf([variableFichero], [textoConFormato], [listaVariablesFormato])
    - Requiere que el fichero esté abierto
    - Funciona como printf

```
fprintf(f, "Hola mundo %d\n", x);
```

- **Cierre:** fclose([variableFichero])

```
fclose(f);
```

Funciones para **trabajar con cadenas** (requieren #include <string.h>):

- **Longitud:** strlen([texto]);

```
char texto[] = "Hola";
int longitud = strlen(texto);
```

- ***Copiar el valor:*** strcpy([origen], [destino]);

```
char origen[] = "Hola";
char destino[20];
strcpy(destino, origen);
```

- ***Comparar valores:*** strcmp ([c1], [c2]);
    - Devuelve un entero:
        - 0 si son iguales
        - > 0 si el primero es mayor
        - < 0 si el segundo es mayor

```
char origen[] = "Hola";
char destino[20];
strcpy(destino, origen);
```

- ***Concatenar valores:*** strcat([destino], [nuevotexto]);

```
char destino[20] = "Hola ";
strcat(destino, "Mundo");
```

Estructuras básicas

- **Condicional**:
  if ([condición]) {
    // Código
  } else if([condición]) {
    // Código
  } else if …
    …
  } else {
    // Código
  }

```
if (numero % 2 == 0) {
    printf("Es par\n");
} else {
    printf("Es impar\n");
}
```

- **Bucle for:**

  for (int [variable] =[valorInicio];[condicion];[incremento]) {
    // Código
  }

```
for (int i = 0; i < 5; i++) {
    printf("i = %d\n", i);
}
```

- **Bucle While**
  while ([condicion]) {
    // Código
  }

```
int i = 0;
while (i < 5) {
    printf("i = %d\n", i);
    i++;
}
```

- **Bucle: Do while**

```
do {
   // Codigo
} while ([CONDICION];
```

```
int i = 0;
do {
   printf("i = %d\n", i);
   i++;
} while (i < 5);
```

Estructura de un programa

La estructura de un programa general debe ser la siguiente:

```
/***** INCLUDES *****/
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
/***** CONSTANTES *****/
#define MAX 100
#define PI 3.1416
/***** TIPOS (struct, enum, typedef) *****/
struct Persona {
    char nombre[30];
    int edad;
};
/***** PROTOTIPOS DE FUNCIONES *****/
int sumar(int a, int b);
void saludar();

/***** FUNCION PRINCIPAL *****/
int main() {
    // Declaración de variables
    int x = 5;
    int y = 3;
    int resultado;
    // Llamadas a funciones
    resultado = sumar(x, y);
    printf("Resultado: %d\n", resultado);
    saludar();
    return 0;
```

```
}


/***** DEFINICION DE FUNCIONES *****/
int sumar(int a, int b) {
    return a + b;
}
void saludar() {
    printf("Hola\n");
}
```

Nota: las definiciones se pueden subir junto con sus prototipos.

Librerías

Las librerías están compuestas de 3 archivos:

- Archivo.h (interfaz, lo que se EXPORTA).
    - Siempre empieza por un #ifndef [NOMBRE], seguido por un #define [NOMBRE] y termina por #endif
        - Para evitar que se importe varias veces

```
#ifndef MILIB_H
#define MILIB_H
/***** CONSTANTES EXPORTADAS *****/
#define PI 3.1416
/***** TIPOS EXPORTADOS *****/
typedef struct {
    int x;
    int y;
} Punto;
/***** PROTOTIPOS EXPORTADOS *****/
int sumar(int a, int b);
void saludar(void);
#endif
```

- Archivo.c (IMPLEMENTACION + IMPORTA)

```
#include <stdio.h>

#include "milib.h"

/***** IMPLEMENTACION *****/

int sumar(int a, int b) {

   return a + b;

}

void saludar(void) {

   printf("Hola desde la libreria\n");

}
```

- Archivo.o (generado por el compilador al compilar)

Compilación

- Para **compilar** un código debemos usar el compilador:

  gcc [ficheroCodigo] -o [ficheroSalida]

  ```
  gcc main.c -o programa
  ```

- Para **ejecutar**, usaremos una terminal:

  ./[ficheroSalida]

  ```
  ./programa
  ```

- Para emplear **librerías**, podemos:
  - Complilación conjunta:

    gcc [ficheroMain] [ficheroLibreria] -o [ficheroSalida]

    ```
    gcc main.c libreria.c -o programa
    ```

  - Compilación por separado:

    gcc [librería]

    gcc [ficheroMain] [libreríaCompilada]

    ```
    gcc libreria.c

    gcc main.c libreria.o -o programa
    ```

- Al compilador se le pueden añadir los siguientes **flags interesantes**:
    - –Wall: Mostrar warnings
    - -Wextra: Mostrar warnings avanzados
    - -o: especificar fichero de salida
    - -g: preparar compilado para debug
    - -lm funciones matemáticas